AquaGuardian - Machine Learning Regression System

📋 Table of Contents

🎯 Overview

The AquaGuardian regression system uses **Multiple Linear Regression with Ridge Regularization** to create an adaptive difficulty system for the diving game. It learns from player performance across trials and automatically adjusts game parameters to maintain an optimal challenge level.

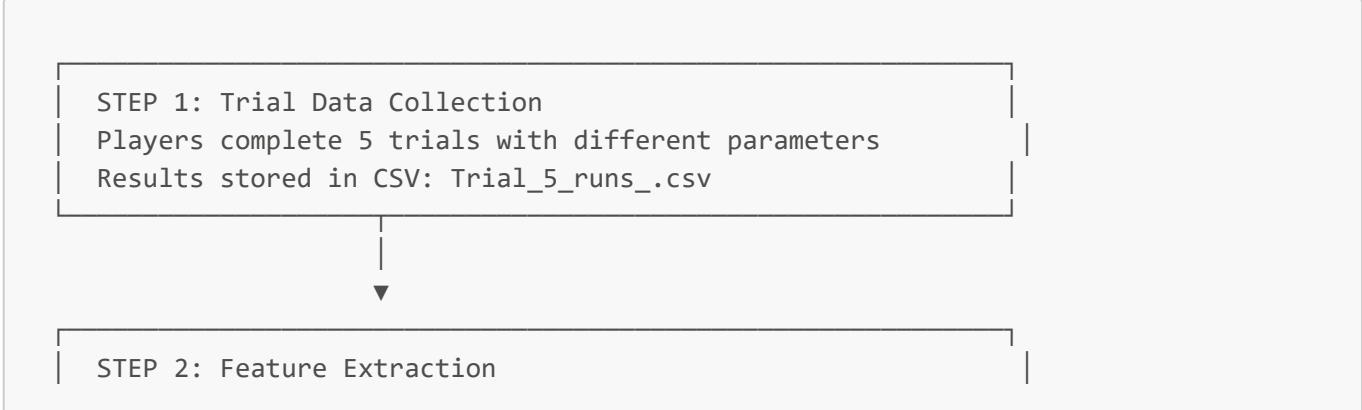
What It Does:

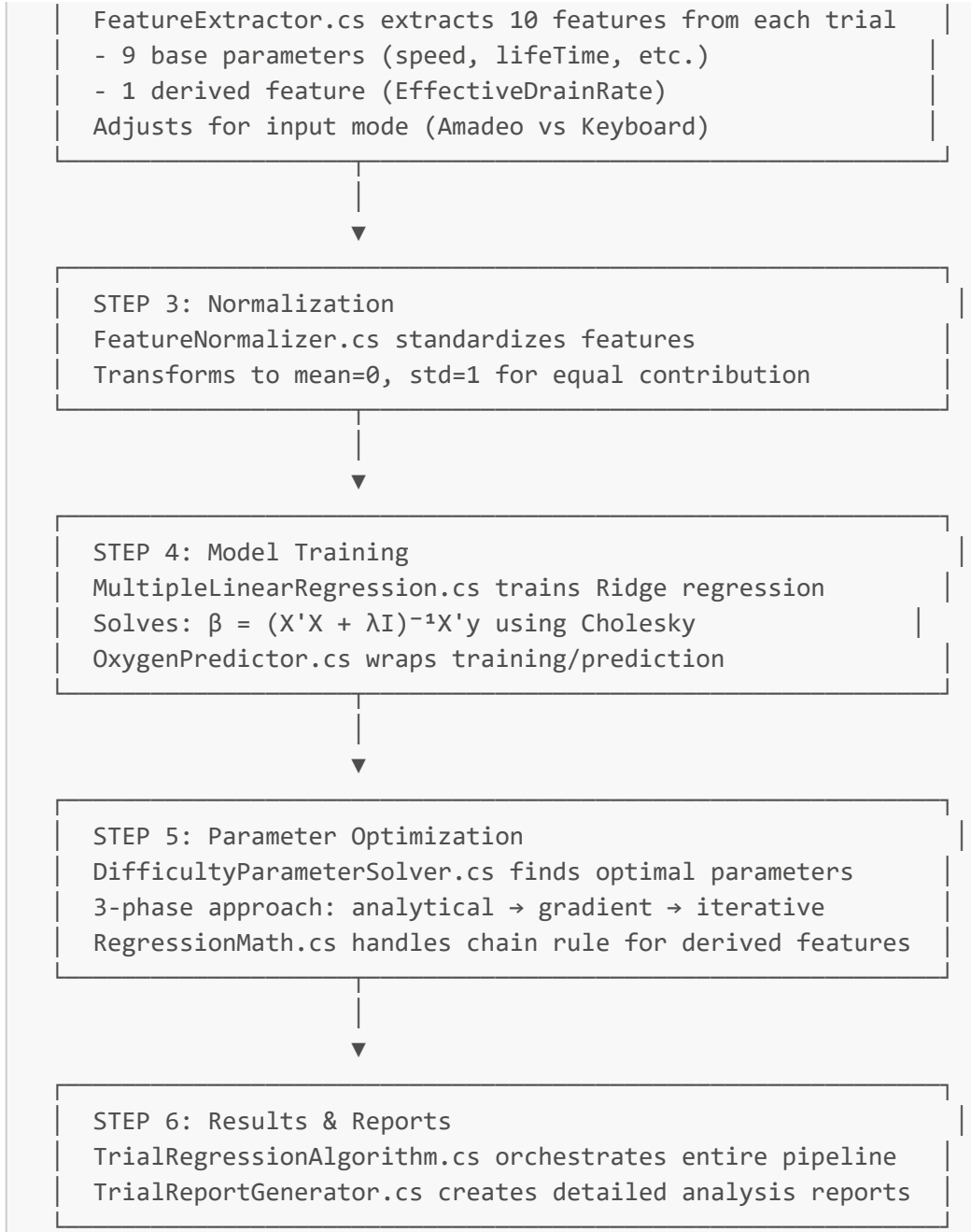
- **Learns** from player trials - as players complete levels with different parameters, the system builds a model of how parameters affect oxygen consumption
- **Predicts** oxygen levels - given any set of game parameters, it can predict how much oxygen the player will have remaining
- **Optimizes** difficulty - finds the perfect parameter combination to achieve a target oxygen level (default: 10%)
- **Personalizes** experience - uses each player's historical data to create customized difficulty settings

---

🏗️ System Architecture

The system follows a pipeline architecture where data flows from trial collection through analysis to optimization:





🔧 Core Components

1. **FeatureExtractor.cs** - The Feature Hub

Location: `Assets/Scripts/Linear regression/FeatureExtractor.cs`

This is the central hub for all feature-related operations. Instead of having feature extraction logic scattered across multiple files, everything goes through here.

What it does:

- Defines the canonical list of 10 features used throughout the system
- Converts raw trial data into feature vectors that the ML model can understand
- Handles input mode differences (Amadeo device vs keyboard controls)
- Calculates personalized baseline parameters from player history

Key Functions:

- `FeatureNames` - The definitive list of all 10 feature names
- `ExtractFeatures(TrialData)` - Converts a single trial into a float array of features
- `ExtractFeaturesAndTargets(List<TrialData>)` - Prepares training data (X matrix and y vector)
- `GetPatientBaseline(trials, ranges, useMedian)` - Calculates personalized starting point for optimization

Why centralize this? Having one place for feature extraction eliminates bugs from inconsistent calculations and makes it easy to add new features or modify existing ones.

---

2. **MultipleLinearRegression.cs** - The ML Engine

Location: `Assets/Scripts/Linear regression/MultipleLinearRegression.cs`

This is the core machine learning model that learns the relationship between game parameters and oxygen consumption.

The Model: The system uses linear regression to model oxygen as a weighted sum of features:

$$\text{Oxygen} = \beta_0 + \beta_1 \times \text{speed} + \beta_2 \times \text{verticalSpeed} + \ldots + \beta_{10} \times \text{EffectiveDrainRate}$$

Each coefficient (β) represents how much that feature affects oxygen. For example, if $\beta_1 = -2.5$, then increasing speed by 1 unit decreases oxygen by 2.5%.

Key Techniques:

Ridge Regularization (λ = 0.5-2.0)

- Prevents overfitting when we have limited trial data (5-10 trials)
- Adds a penalty for large coefficients, keeping the model stable
- Lambda adapts based on sample size: fewer trials = more regularization

Cholesky Decomposition

- Efficient method to solve the regression equation: $\beta = (X'X + \lambda I)^{-1}X'y$
- More stable than direct matrix inversion
- Takes advantage of the fact that X'X is symmetric and positive definite

Normalization

- All features are standardized to mean=0, std=1 before training
- Ensures features on different scales contribute equally
- Makes optimization more stable

Training Process:

1. Normalize features using FeatureNormalizer
2. Build design matrix with intercept column

3. Solve normal equations using Cholesky decomposition
4. Calculate performance metrics (R^2, RMSE, MAE)

---

3. **OxygenPredictor.cs** - The User-Friendly Wrapper

Location: `Assets/Scripts/Linear regression/OxygenPredictor.cs`

This wraps the MultipleLinearRegression model with a simpler interface and adds smart features for small datasets.

What it does:

- Simplifies training - just pass in a list of trials
- Handles feature selection automatically for small datasets
- Applies adaptive regularization based on sample size
- Provides easy prediction interface

Key Methods:

- `TrainModel(trials, enableFeatureSelection)` - Trains the model with automatic configuration
- `PredictOxygen(parameters)` - Predicts oxygen level (clamped to 0-100%)
- `PredictOxygenUnclamped(parameters)` - Raw prediction without clamping
- `GetFeatureImportance()` - Shows which features matter most

Adaptive Feature Selection: When you have fewer than 10 trials, using all 10 features risks overfitting. The predictor automatically:

1. Trains a temporary model on all features
2. Ranks features by importance (absolute coefficient value)
3. Selects top K features (where K = maxFeaturesForTraining)
4. Retrains using only those features

This reduces overfitting risk while keeping the most important relationships.

Adaptive Regularization:

```
λ = Clamp(0.5 + (10 - trialCount) × 0.2, 0.5, 2.0)
```

Fewer trials → stronger regularization → more stable predictions

---

4. **DifficultyParameterSolver.cs** - The Optimization Engine

Location: `Assets/Scripts/Linear regression/DifficultyParameterSolver.cs`

This is where the magic happens - finding the perfect game parameters to hit a target oxygen level (usually 10%).

The Challenge: Given a trained model, we need to find parameter values that will result in exactly 10% oxygen remaining. This is an inverse problem: instead of predicting oxygen from parameters, we're finding parameters from desired oxygen.

Three Optimization Methods (in order of preference):

Method 1: `SolveForTargetOxygen()` ⭐ **PRIMARY METHOD**

This is the main workhorse - a sophisticated 3-phase optimizer:

Phase 1: Analytical Minimal-Change Solution

- Calculates the smallest parameter adjustments needed to reach the target
- Works in normalized feature space for numerical stability
- Formula: $\Delta\hat{x} = (target - fixed) \times a / ||a||^2$
- Typically gets within 1-2% of target

Phase 2: Gradient Refinement

- Uses projected gradient descent to fine-tune the solution
- Adaptive learning rate based on how far we are from target
- Respects parameter boundaries (min/max constraints)
- Usually achieves < 0.5% error

Phase 3: Iterative Refinement (if needed)

- Extended gradient descent with best-solution tracking
- Only runs if error is still > 0.5% after phase 2
- More iterations, more careful step sizes
- Typically achieves < 0.1% error

Method 2: `RandomSweepOptimizer()` **FALLBACK**

- Used when gradient methods fail or error > 5%
- Generates 150 random parameter combinations
- Tests each one and returns the best
- Slower but more robust for difficult cases

Method 3: `SolveForTargetDifficultyMulti()` **FINAL FALLBACK**

- Gradient descent on top K features
- Simpler than Method 1 but less accurate
- Guaranteed to return a solution
- Used as last resort if others fail

Chain Rule Magic: When optimizing `lifeTime` or `RemoveHealthEveryLifeTime`, the solver must account for their effect on the derived feature `EffectiveDrainRate`. This is handled by RegressionMath.EffectiveBeta():

```
β_effective = β_base + β_derived × (∂derived/∂base) × (σ_base / σ_derived)
```

For example, changing `lifeTime` affects oxygen both directly AND through its effect on drain rate.

---

5. **RegressionMath.cs** - Mathematical Utilities

Location: `Assets/Scripts/Linear regression/RegressionMath.cs`

This file contains the mathematical heavy lifting for optimization, particularly the chain rule calculations for derived features.

The Chain Rule Problem: `EffectiveDrainRate` is calculated as `RemoveHealthEveryLifeTime / lifeTime`. When we optimize `lifeTime`, we're not just changing one feature - we're also changing the drain rate. The model has coefficients for both, so we need to account for both effects.

Key Functions:

`EffectiveBeta(model, featureIndex, params, optimizingDerivedDependencies)` Calculates the true coefficient for a feature, including its indirect effects:

For `lifeTime` (index 3):

```
// Direct effect
β_direct = model.coefficients[4]  // coefficient for lifeTime

// Indirect effect through EffectiveDrainRate
∂EDR/∂lifeTime = -RemoveHealthEveryLifeTime / lifeTime²
β_indirect = β_EDR × ∂EDR/∂lifeTime × (σ_lifeTime / σ_EDR)

// Total effect
β_effective = β_direct + β_indirect
```

For `RemoveHealthEveryLifeTime` (index 4):

```
∂EDR/∂drop = 1 / lifeTime
β_effective = β_drop + β_EDR × (1/lifeTime) × (σ_drop / σ_EDR)
```

`SumBetaSq(model, freeFeatures, params)` Calculates the sum of squared effective betas for normalization in gradient descent. This ensures all features contribute proportionally to their importance.

Other Utilities:

- `SolveSPDByCholesky(A, b)` - Solves linear systems using Cholesky decomposition
- `ComputeMetrics(yTrue, yPred)` - Calculates R^2, RMSE, MAE
- `CleanMatrix(X)` / `CleanVector(y)` - Replaces NaN/Inf with safe values
- `ValidateInputs(X, Y)` - Checks data validity before training

---

6. **ParameterHelper.cs** - Unified Parameter Access

Location: `Assets/Scripts/Linear regression/ParameterHelper.cs`

A utility class that provides index-based access to trial parameters. Instead of writing `trial.speed`, `trial.verticalSpeed`, etc. everywhere, you can use `ParameterHelper.Get(trial, 0)`, `ParameterHelper.Get(trial, 1)`, etc.

Why use indices? When optimizing multiple parameters in a loop, it's much cleaner to iterate over indices than to have a giant switch statement for each parameter name.

Key Functions:

`Get(TrialData, index)` **and** `Set(ref TrialData, index, value)` Access parameters by index (0-9 for base parameters, 9-10 for derived):

```
float speed = ParameterHelper.Get(trial, 0);
ParameterHelper.Set(ref trial, 0, 25.0f);
```

`Range(ParameterRanges, index)` Returns the valid (min, max) range for a parameter:

```
var (min, max) = ParameterHelper.Range(ranges, 3);  // lifeTime range
```

`HasHeadroom(index, ranges, params, threshold)` Checks if a parameter has room to move (not stuck at min or max):

```
bool canAdjust = ParameterHelper.HasHeadroom(3, ranges, trial, 0.1f);
```

`Clamp(index, value, bounds)` Clamps a parameter value to its valid range.

`Clone(TrialData)` Creates a deep copy of trial parameters.

Index Mapping:

```
0: speed                    1: verticalSpeed
2: idleUpwardSpeed          3: lifeTime
4: RemoveHealthEveryLifeTime  5: removeHealthWithCollide
6: timeBetweenCollides      7: healHealthPoint
8: factorForce              9: EffectiveDrainRate (derived, read-only)
```

Feature Engineering

Feature Definitions (10 Total)

The system extracts 10 features from each trial - 9 base parameters that you can directly control, plus 1 derived feature that captures an important relationship.

Base Features (9):

1. **speed** - How fast the player moves forward through the cave
2. **verticalSpeed** - How fast the player moves up/down
3. **idleUpwardSpeed** - Passive upward drift when not moving (×0.5 in Amadeo mode for better control)
4. **lifeTime** - Seconds between automatic oxygen drain cycles
5. **RemoveHealthEveryLifeTime** - How much oxygen is removed each cycle
6. **removeHealthWithCollide** - Oxygen lost when hitting walls
7. **timeBetweenCollides** - Cooldown between collision damage (seconds)
8. **healHealthPoint** - Oxygen restored by collecting health packs
9. **factorForce** - Force multiplier for Amadeo device (0 in keyboard mode)

Derived Feature (1):

10. **EffectiveDrainRate** = `RemoveHealthEveryLifeTime / lifeTime`
    - This captures the actual drain rate per second
    - For example: removing 3 health every 2 seconds = 1.5 health/second
    - The model can learn that it's the rate that matters, not the individual values
    - Automatically recalculated whenever `lifeTime` or `RemoveHealthEveryLifeTime` changes

Note: In earlier versions, we also had `EffectiveCollisionDamageRate`, but testing showed it was redundant and removing it improved model performance.

Input Mode Adjustments

The system handles two input modes differently:

Keyboard Mode (IsAmadeoMode = 0):

- `factorForce` is set to 0 (not used)
- `idleUpwardSpeed` used as-is

Amadeo Mode (IsAmadeoMode = 1):

- `factorForce` uses actual value from device
- `idleUpwardSpeed` multiplied by 0.5 (weaker drift for better control)

Feature Normalization

Before training, all features are normalized using **Z-score standardization**:

```
x_normalized = (x - μ) / σ
```

Where:

- `μ` = mean of that feature across all trials

- σ = standard deviation (using sample std with n-1)

Why normalize? Without normalization, features with larger scales dominate the model. For example, `speed` (range: 10-40) would have much more influence than `lifeTime` (range: 0.5-4) just because the numbers are bigger. Normalization puts everything on the same scale so each feature's importance is determined by its actual relationship to oxygen, not its numerical range.

📊 Regression Model

Model Equation

The model predicts oxygen as a linear combination of the 10 features:

$$\begin{aligned}\text{Oxygen} = {} & \beta_0 + \beta_1 \times \text{speed} + \beta_2 \times \text{verticalSpeed} + \beta_3 \times \text{idleUpwardSpeed} + \\ & \beta_4 \times \text{lifeTime} + \beta_5 \times \text{RemoveHealthEveryLifeTime} + \\ & \beta_6 \times \text{removeHealthWithCollide} + \beta_7 \times \text{timeBetweenCollides} + \\ & \beta_8 \times \text{healHealthPoint} + \beta_9 \times \text{factorForce} + \\ & \beta_{10} \times \text{EffectiveDrainRate} + \varepsilon\end{aligned}$$

Where:

- β_0 = intercept (baseline oxygen level)
- $\beta_1 \ldots \beta_{10}$ = coefficients (learned from training data)
- ε = error term (unexplained variance)

Reading the coefficients:

- Positive β means increasing that feature increases oxygen (good for player)
- Negative β means increasing that feature decreases oxygen (harder for player)
- Larger |β| means that feature has more impact

Ridge Regression

The system uses Ridge Regression, which adds a penalty for large coefficients to prevent overfitting:

Optimization Problem:

$$\underbrace{\|y - X\beta\|^2}_{\text{fit to data}} + \underbrace{\lambda\|\beta\|^2}_{\text{regularization}}$$

Closed-Form Solution:

$$\beta = (X'X + \lambda I)^{-1}X'y$$

Where:

- `λ` = regularization strength (0.5-2.0, adaptive based on sample size)
- `I` = identity matrix
- `X'X + λI` is always invertible and positive definite

Why Ridge? With only 5-10 trials and 10 features, ordinary least squares would overfit. Ridge regression adds a penalty that keeps coefficients reasonable, leading to better predictions on new data. The penalty doesn't apply to the intercept β_0.

Model Metrics

- **R^2 (Coefficient of Determination):** Proportion of variance explained
  - Range: [-∞, 1]
  - $R^2 = 1$ → perfect fit
  - $R^2 = 0$ → model no better than mean
- **Adjusted R^2:** R^2 adjusted for number of features
  - Penalizes extra features
  - Formula: `1 - (1-R²)×(n-1)/(n-p-1)`
- **RMSE (Root Mean Squared Error):** Average prediction error
  - Formula: `√(Σ(y_pred - y_actual)² / n)`
  - Lower is better
- **MAE (Mean Absolute Error):** Average absolute error
  - Formula: `Σ|y_pred - y_actual| / n`
  - More robust to outliers than RMSE

🎯 Parameter Optimization

Goal

Find game parameters that result in **target oxygen level** (default: 10%).

Entry Point: Where Optimization Starts

Location: `Assets/Scripts/TrialRegressionAlgorithm.cs` (lines 180-188)

```
optimizedSolution = DifficultyParameterSolver.SolveForTargetOxygen(
    model,
    baseParams,
    topIndices,
    targetOxygen,
    ranges,
    d => predictor.PredictOxygen(d),
    out optimizedError);
```

This is called from `PerformRegressionAnalysis()` after training the model.

How Optimal Parameters Are Calculated - Step by Step

Step 1: Get Baseline Parameters

Location: `Assets/Scripts/TrialRegressionAlgorithm.cs` (line 177)

```
var baseParams = FeatureExtractor.GetPatientBaseline(allTrialData, ranges,
useMedian: false);
```

What happens:

- If ≥ 3 trials: Calculates **average** of player's previous trials (personalized starting point)
- If < 3 trials: Falls back to **mid-range** defaults from `ParameterRanges`
- Uses median for `factorForce` if any Amadeo trials exist

Code: `Assets/Scripts/Linear regression/FeatureExtractor.cs` (lines 117-168)

Step 2: Select Features to Optimize

Location: `Assets/Scripts/TrialRegressionAlgorithm.cs` (lines 159-173)

```
var featureImportance = predictor.GetFeatureImportance();
var featuresForOptimization = featureImportance.ToList();

// Filter out factorForce if keyboard-only mode
if (!anyAmadeoTrials)
{
    featuresForOptimization = featuresForOptimization
        .Where(f => f.Item1 != "factorForce")
        .ToList();
}

int[] topIndices = topFeatures.Select(f => System.Array.IndexOf(featureNames,
f.Item1)).ToArray();
```

What happens:

1. Gets feature importance (sorted by |coefficient|)
2. Filters out `factorForce` if keyboard mode
3. Converts feature names to indices (0-10)

Further filtering happens in: `Assets/Scripts/Linear regression/DifficultyParameterSolver.cs` (lines 357-372)

```
int[] free = topFeatureIndices
    .Where(j => j != 9 && j != 10)  // Exclude derived features
    .Where(j => ParameterHelper.HasHeadroom(j, ranges, baseParams, threshold:
0.1f))  // Not at boundary
    .Where(j => Mathf.Abs(model.coefficients[j + 1]) > 1e-6f)  // Beta not
negligible
    .ToArray();
```

Exclusions:

- Derived features (indices 9, 10) - can't optimize directly
- Features at boundaries (no room to adjust)
- Features with near-zero coefficients

Step 3: Solve Minimal-Change Solution 📍 CORE ALGORITHM

Location: `Assets/Scripts/Linear regression/DifficultyParameterSolver.cs` (lines 395-463)

Function: `SolveMinimalChange()`

3.1: Calculate Fixed Contribution

Lines: 405-424

```
float fixedContribution = model.coefficients[0]; // intercept

for (int j = 0; j < 11; j++)  // All 11 features
{
    if (System.Array.IndexOf(freeFeatures, j) >= 0) continue; // skip free ones

    // Skip derived features if optimizing their dependencies
    if (j == 9 && optimizingDrainRateDeps) continue;
    if (j == 10 && optimizingCollisionDeps) continue;

    float xRaw = ParameterHelper.Get(work, j);
    float xHat = model.ToNormalized(j, xRaw);  // Convert to normalized space
    fixedContribution += model.coefficients[j + 1] * xHat;
}
```

What this does:

- Sums contribution from all **locked** features (not being optimized)
- Works in **normalized space** (model coefficients are normalized)
- Skips derived features if their dependencies are being optimized

Mathematical meaning:

```
fixed = β₀ + Σ(βᵢ × x̂ᵢ) for all locked features i
```

3.2: Build Beta Vector for Free Features

Lines: 428-436

```
var a = new float[m];  // m = number of free features

for (int k = 0; k < m; k++)
{
    a[k] = RegressionMath.EffectiveBeta(model, freeFeatures[k], work,
                                        optimizingDrainRateDeps ||
optimizingCollisionDeps);
}
```

What this does:

- Gets effective beta for each free feature
- **Includes chain rule derivatives** if optimizing features that affect derived features
- Handled by `RegressionMath.EffectiveBeta()` - see below

Chain Rule Implementation: `Assets/Scripts/Linear regression/RegressionMath.cs` (lines 14-86)

Example for `lifeTime` (index 3):

```
if (featureIndex == 3) // lifeTime
{
    float currentDrop = ParameterHelper.Get(currentParams, 4);
    float currentLife = ParameterHelper.Get(currentParams, 3);
    if (currentLife > 0.1f)
    {
        // d(EDR)/d(lifeTime) = -drop / life²
        float dEDR_dLife = -currentDrop / (currentLife * currentLife);
        float stdDevLife = model.Stds?.ElementAtOrDefault(3) ?? 1.0f;
        float stdDevEDR = model.Stds?.ElementAtOrDefault(9) ?? 1.0f;
        float normalizedDerivative = dEDR_dLife * stdDevLife / Mathf.Max(1e-6f,
stdDevEDR);
        beta += effectiveDrainRateBeta * normalizedDerivative;
    }
}
```

Mathematical formula:

```
β_effective = β_base + β_derived × (d(derived)/d(base)) × (σ_base / σ_derived)
```

3.3: Calculate Minimal-Change Solution

Lines: 438-445

```
float rhs = targetO2 - fixedContribution;  // Target - fixed = what we need from
free features

// Calculate denominator: ||a||²
double denom = 1e-6;  // Small epsilon to prevent division by zero
for (int k = 0; k < m; k++)
    denom += (double)a[k] * (double)a[k];

float scale = (float)(rhs / denom);  // Scale factor
```

Mathematical solution:

```
We want: a · Δx̂ = target - fixed

Minimal-change solution: Δx̂ = (target - fixed) × a / ||a||²

Where:
- a = vector of effective betas for free features
- ||a||² = sum of squared betas
- scale = (target - fixed) / ||a||²
```

3.4: Apply Changes and Convert to Raw Space

Lines: 447-457

```
for (int k = 0; k < m; k++)
{
    int j = freeFeatures[k];
    float x0_raw = ParameterHelper.Get(work, j);              // Current raw value
    float x0_hat = model.ToNormalized(j, x0_raw);             // Convert to
normalized
    float x1_hat = x0_hat + scale * a[k];                    // Apply change in
normalized space
    float x1_raw = model.FromNormalized(j, x1_hat);          // Convert back to
raw space
    x1_raw = ParameterHelper.Clamp(j, x1_raw, bounds);       // Clamp to valid
range
    ParameterHelper.Set(ref work, j, x1_raw);                 // Update parameter
}
```

What this does:

1. Gets current parameter value (raw)
2. Converts to normalized space: `x̂ = (x - μ) / σ`
3. Applies change: `x̂_new = x̂_old + scale × β`
4. Converts back: `x_new = x̂_new × σ + μ`
5. Clamps to valid range: `min ≤ x ≤ max`

Why work in normalized space?

- Model coefficients are trained on normalized features
- Ensures all features contribute equally
- More numerically stable

Step 4: Iterative Refinement 📍

Location: `Assets/Scripts/Linear regression/DifficultyParameterSolver.cs` (lines 378-390)

```
// 1) Closed-form minimal-change solution
var x0 = SolveMinimalChange(model, baseParams, free, targetO2, bounds);

// 2) Projected-gradient refine (50 steps, learning rate 0.3)
var x1 = RefineProjectedGradient(model, x0, free, targetO2, bounds, predictO2,
                                 maxSteps: 50, lr: 0.3f, tol: 0.01f);

float o1 = predictO2(x1);
finalError = Mathf.Abs(o1 - targetO2);

// 3) If still not close enough, try iterative refinement (100 iterations)
if (finalError > 1.0f)
{
    x1 = RefineProjectedGradientIterative(model, x1, free, targetO2, bounds,
predictO2,
                                          maxIterations: 100);
}
```

Refinement Function: `RefineProjectedGradient()` (lines 465-530)

Algorithm:

```
for (int step = 0; step < maxSteps; step++)
{
    float y = predictO2(cur);                    // Current prediction
    float error = y - targetO2;                  // Error
    if (Mathf.Abs(error) <= tol) break;          // Converged

    // Calculate gradient
    float sumBetaSq = RegressionMath.SumBetaSq(model, freeFeatures, cur);

    foreach (int j in freeFeatures)
```

```
    {
        float betaEff = RegressionMath.EffectiveBeta(model, j, cur, ...);
        float grad = betaEff * error / Mathf.Max(sumBetaSq, 1e-6f);

        // Update in normalized space
        float xNorm = xRaw.ToNormalized(j, model);
        float newNorm = xNorm + stepSize * grad;
        float newRaw = newNorm.FromNormalized(j, model);

        // Project to feasible region
        newRaw = ParameterHelper.Clamp(j, newRaw, bounds);
        ParameterHelper.Set(ref cur, j, newRaw);
    }
}
```

What this does:

- Refines solution using **projected gradient descent**
- Updates parameters in direction that reduces error
- Projects to feasible region (clamps to bounds)
- Stops when error < tolerance or max steps reached

Complete Optimization Flow

```
1. Get Baseline (FeatureExtractor.GetPatientBaseline)
   ↓
2. Select Features (Filter by importance, exclude derived)
   ↓
3. Solve Minimal-Change (SolveMinimalChange)
   ├─ Calculate fixed contribution (locked features)
   ├─ Build beta vector (with chain rule)
   ├─ Solve: Δx̂ = (target - fixed) × a / ||a||²
   └─ Convert to raw space & clamp
   ↓
4. Refine (RefineProjectedGradient)
   ├─ Calculate error
   ├─ Update parameters: x_new = x_old + lr × gradient
   ├─ Project to bounds
   └─ Repeat until convergence
   ↓
5. Return Optimized Parameters
```

Chain Rule for Derived Features

When optimizing `lifeTime` (index 3), must account for `EffectiveDrainRate` (index 9):

```
EffectiveDrainRate = RemoveHealthEveryLifeTime / lifeTime

d(EDR)/d(lifeTime) = -RemoveHealthEveryLifeTime / lifeTime²

β_effective = β_lifeTime + β_EDR × (-drop/life²) × (σ_lifeTime / σ_EDR)
```

Code Location: `Assets/Scripts/Linear regression/RegressionMath.cs` (lines 28-40)

Similar logic applies for:

- `RemoveHealthEveryLifeTime` → affects `EffectiveDrainRate` (lines 42-54)
- `removeHealthWithCollide` → affects `EffectiveCollisionDamageRate` (lines 57-68)
- `timeBetweenCollides` → affects `EffectiveCollisionDamageRate` (lines 70-82)

Key Files and Line Numbers

Step	File	Lines	Function
Entry Point	`TrialRegressionAlgorithm.cs`	180-188	`SolveForTargetOxygen()` call
Baseline	`FeatureExtractor.cs`	117-168	`GetPatientBaseline()`
Feature Selection	`TrialRegressionAlgorithm.cs`	159-173	Feature filtering
Feature Filtering	`DifficultyParameterSolver.cs`	357-372	Mobile feature selection
Minimal-Change	`DifficultyParameterSolver.cs`	395-463	`SolveMinimalChange()`
Fixed Contribution	`DifficultyParameterSolver.cs`	405-424	Calculate locked features
Beta Vector	`DifficultyParameterSolver.cs`	428-436	Build beta array
Chain Rule	`RegressionMath.cs`	14-86	`EffectiveBeta()`
Solution	`DifficultyParameterSolver.cs`	438-445	Calculate scale
Apply Changes	`DifficultyParameterSolver.cs`	447-457	Update parameters
Refinement	`DifficultyParameterSolver.cs`	465-530	`RefineProjectedGradient()`

Step	File	Lines	Function
Iterative	DifficultyParameterSolver.cs	532-583	RefineProjectedGradientIterative()

How It Works

Training Pipeline

```
1. Load Trial Data (CSV)
   ↓
2. Extract Features (FeatureExtractor.ExtractFeatures)
   - Convert TrialData → float[11]
   - Adjust for input mode (Amadeo vs Keyboard)
   - Calculate derived features
   ↓
3. Normalize Features (FeatureNormalizer)
   - Calculate μ, σ for each feature
   - Transform: x_norm = (x - μ) / σ
   ↓
4. Train Model (MultipleLinearRegression.Fit)
   - Build design matrix X (n×11)
   - Solve: β = (XᵀX + λI)⁻¹Xᵀy
   - Calculate R², RMSE, MAE
   ↓
5. Validate Model
   - Check R² > threshold
   - Check MAE < threshold
   - Check variance in data
```

Prediction Pipeline

```
1. Extract Features (FeatureExtractor.ExtractFeatures)
   ↓
2. Normalize (using trained normalizer)
   ↓
3. Predict: Oxygen = β₀ + Σ(βᵢ × xᵢ)
   ↓
4. Clamp to [0, 100]%
```

Optimization Pipeline

See detailed explanation in Parameter Optimization section above.

```
1. Get Baseline Parameters
   - Average/Median of player's trials (if ≥ 3) ←
```

```
FeatureExtractor.GetPatientBaseline()
   - Mid-range defaults (if < 3)
   ↓
2. Select Features to Optimize
   - Top K by importance ← TrialRegressionAlgorithm.cs (159-173)
   - Exclude derived features ← DifficultyParameterSolver.cs (357-372)
   ↓
3. Solve Minimal-Change Solution ← DifficultyParameterSolver.SolveMinimalChange()
(395-463)
   ├─ Calculate fixed contribution (locked features) ← Lines 405-424
   ├─ Build beta vector (with chain rule) ← Lines 428-436,
RegressionMath.EffectiveBeta()
   ├─ Solve: Δx̂ = (target - fixed) × a / ||a||² ← Lines 438-445
   └─ Convert to raw space & clamp ← Lines 447-457
   ↓
4. Iterative Refinement ← DifficultyParameterSolver.RefineProjectedGradient()
(465-530)
   ├─ Calculate error
   ├─ Update parameters: x_new = x_old + lr × gradient
   ├─ Project to bounds
   └─ Repeat until convergence (50 steps, then 100 if needed)
   ↓
5. Return Optimized Parameters
```

Key Algorithms

1. Minimal-Change Solver

Problem: Find $\Delta\hat{x}$ that minimizes $||\Delta\hat{x}||^2$ subject to `a·Δx̂ = target - fixed`

Solution:

```
Δx̂ = (target - fixed) × a / ||a||²
```

Where:

- `a` = vector of betas for free features
- `fixed` = contribution from locked features
- `target` = target oxygen level

Why Minimal-Change?

- Minimizes deviation from baseline
- Preserves player's preferred parameter ranges
- More personalized solution

2. Chain Rule Derivatives

Problem: Derived features depend on base features, so optimizing base features affects derived features.

Solution: Add derivative contribution to beta:

```
β_effective = β_base + β_derived × (d(derived)/d(base)) × (σ_base / σ_derived)
```

Example: Optimizing `lifeTime` affects `EffectiveDrainRate`:

```
β_eff = β_lifeTime + β_EDR × (-drop/life²) × (σ_lifeTime / σ_EDR)
```

3. Projected Gradient Descent

Problem: Gradient descent might violate constraints (min/max bounds).

Solution: Project gradient to feasible region:

```
1. Calculate gradient: g = β × error / ||β||²
2. Update: x_new = x_old + stepSize × g
3. Project: x_new = clamp(x_new, min, max)
4. Repeat until convergence
```

File Structure

Core ML Files

```
Assets/Scripts/Linear regression/
├── FeatureExtractor.cs          # Feature definitions & extraction (CENTRAL HUB)
├── FeatureNormalizer.cs          # Z-score normalization
├── MultipleLinearRegression.cs  # Ridge regression with Cholesky solver
├── OxygenPredictor.cs           # Model wrapper (train/predict)
├── DifficultyParameterSolver.cs # Parameter optimization (3 methods)
├── RegressionMath.cs            # Chain rule derivatives
├── ParameterHelper.cs           # Parameter access by index
└── OxygenCalculationSettings.cs  # Oxygen calculation mode settings
```

Data & Orchestration

```
Assets/Scripts/
├── TrialDataModels.cs           # Data structures (TrialData, ParameterRanges)
├── TrialRegressionAlgorithm.cs  # Main orchestrator (trains model, runs
optimization)
├── TrialReportGenerator.cs      # Generates regression reports
└── Trial/
    └── TrialParameterManager.cs # Loads/saves trial data from CSV
```

Data Files

```
Assets/Data/Trials/
├── Trial_5_runs_.csv            # Constant parameters (CSV mode)
└── Trial_Random_Parameters.csv  # Random parameters (Random mode)
```

💻 Usage Examples

Example 1: Train Model and Predict

```
// Load trial data
List<TrialDataModels.TrialData> trials = LoadTrialsFromCSV();

// Train model
var predictor = new OxygenPredictor();
bool trained = predictor.TrainModel(trials, enableFeatureSelection: false);

if (trained)
{
    // Predict oxygen for new parameters
    var testParams = new TrialDataModels.TrialData
    {
        speed = 25f,
        verticalSpeed = 30f,
        lifeTime = 2f,
        // ... other parameters
    };

    float predictedOxygen = predictor.PredictOxygen(testParams);
    Debug.Log($"Predicted oxygen: {predictedOxygen:F2}%");
}
```

Example 2: Find Optimal Parameters

```
// Train model
var predictor = new OxygenPredictor();
predictor.TrainModel(trials);

// Find parameters for 10% oxygen
var optimal = predictor.FindOptimalParameters(targetOxygen: 10f);

Debug.Log($"Optimal speed: {optimal.speed:F2}");
Debug.Log($"Optimal lifeTime: {optimal.lifeTime:F2}");
```

Example 3: Full Regression Analysis

```
// Run complete analysis
var result = TrialRegressionAlgorithm.PerformRegressionAnalysis(
    allTrialData,
    targetOxygen: 10f
);

// Access results
Debug.Log($"R²: {result.correlations}");
Debug.Log($"Optimized solution: {result.optimizedSolution}");
Debug.Log($"Error: {result.optimizedSolutionError:F3}%");

// Save report
TrialRegressionAlgorithm.SaveRegressionResultsToFile(result);
```

Example 4: Use Centralized Feature Extraction

```
// Extract features from single trial
float[] features = FeatureExtractor.ExtractFeatures(trialData);

// Extract features from multiple trials
float[][] X = FeatureExtractor.ExtractFeatures(trials);
float[] y = trials.Select(t => t.finalOxygenRemaining).ToArray();

// Or use convenience method
var (X, y) = FeatureExtractor.ExtractFeaturesAndTargets(trials);
```

🔍 Key Design Decisions

1. **Centralized Feature Extraction**

- **Why:** Eliminates duplication, ensures consistency
- **Where:** `FeatureExtractor.cs` is single source of truth
- **Benefit:** Easy to add/modify features

2. **Derived Features as Properties**

- **Why:** Always up-to-date, no manual recalculation
- **Implementation:** C# properties calculate on access
- **Benefit:** Automatic dependency handling

3. **Normalized Space Optimization**

- **Why:** Features have different scales, normalization equalizes contribution
- **Implementation:** Optimize in normalized space, convert to raw space
- **Benefit:** More stable optimization

4. **Personalized Baseline**

- **Why:** Each player has different skill level
- **Implementation:** Use median of player's trials as starting point
- **Benefit:** More accurate optimization

5. **Chain Rule for Derived Features**

- **Why:** Derived features affect optimization gradients
- **Implementation:** Add derivative contribution to beta coefficients
- **Benefit:** Correct optimization when optimizing base features

---

Model Performance

Typical Metrics

- **R^2:** 0.85 - 0.99 (good to excellent)
- **RMSE:** 2 - 8% (acceptable)
- **MAE:** 1.5 - 6% (acceptable)

Factors Affecting Performance

1. **Number of Trials:** More trials → better model
2. **Variance in Data:** Need diversity in parameters and outcomes
3. **Feature Selection:** Using all 11 features usually best
4. **Ridge Lambda:** λ = 0.5 prevents overfitting

Troubleshooting

Issue: "Not enough variance in data"

Cause: All trials have similar oxygen levels
Solution: Ensure parameter diversity across trials

Issue: Poor R^2 (< 0.5)

Cause: Model not capturing relationships
Solution: Check feature importance, consider more trials

Issue: Optimization gives wrong oxygen

Cause: Chain rule not applied correctly
Solution: Verify `RegressionMath.EffectiveBeta()` is called

Issue: factorForce appears in keyboard mode

Cause: Not filtered from optimization
Solution: Check `anyAmadeoTrials` flag in solver

References

- **Ridge Regression:** Hoerl & Kennard (1970)
- **Cholesky Decomposition:** Golub & Van Loan (2013)
- **Chain Rule:** Standard calculus for composite functions
- **Projected Gradient Descent:** Boyd & Vandenberghe (2004)

Notes

- Model assumes **linear relationships** between features and oxygen
- **Derived features** are calculated automatically (no CSV storage needed)
- **Normalization** ensures all features contribute equally
- **Personalization** uses median (robust to outliers) instead of mean
- **Chain rule** ensures correct optimization when base features affect derived features

---

Last Updated: 2025-01-XX
Author: AquaGuardian Development Team